АЛМАТИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ

Кафедра информационных систем

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №11

Проектирование стратегии шардирования и тест распределения

Дисциплина: «Проектирование и применение NoSQL-технологий»

3 курс, специальность «Информационные системы»

Вариант 5. Библиотека: кандидат readerId; альтернативный низкокардинальный ключ genreId; основной запрос — выдачи читателя.

Студент: _____ Группа: _____

Преподаватель: _____

Алматы, 2026

1. Цель работы и предметная область

Цель: спроектировать стратегию горизонтального масштабирования документного хранилища (MongoDB), выбрать shard key на основе Access Patterns, количественно проверить распределение документов и нагрузки на синтетических данных, выявить горячие шарды и обосновать архитектурное решение.

Предметная область (вариант 5). Библиотечный сервис хранит коллекцию `loans` — выдачи книг. Документ: `{readerId, genreId, loanId, day, ...}`. Основной сценарий — получить историю выдач конкретного читателя. Модель: 12 000 документов, 30 000 запросов, 3 шарда (A, B, C), 2 500 уникальных readerId (требование: не менее 1000), 5 жанров, 30 дневных «корзин» day.

Шардирование делит набор данных между шардами, репликация лишь копирует данные ради доступности. Запросы принимает маршрутизатор mongos и по shard key направляет их на один или несколько шардов.

2. Access Patterns

AP	Описание	Частота	Фильтр / сортировка
AP-01	Последние выдачи конкретного читателя (основной)	Очень высокая, ~55%	readerId = X, sort loanDate desc, limit 20
AP-02	Выдачи за интервал времени	Средняя, ~10%	loanDate in [t1, t2]
AP-03	Агрегация выдач по жанру (отчёт)	Низкая, ~5%	group by genreId, count
AP-04	Загрузка нового потока выдач	Постоянная, запись	insert, loanId растёт
AP-05	Просроченные выдачи читателя	Средняя, ~15%	readerId = X, dueDate < now
AP-06	Популярные книги жанра за неделю	Низкая, ~5%	genreId = G, day in неделя

Доминирует AP-01 и AP-05 (фильтр по readerId), поэтому ключ должен содержать readerId, чтобы эти запросы были targeted.

3. Кандидаты shard key

Кандидат	Тип	Кардинальность	Монотонность	Оценка
(A) genreId	ranged/по модулю	5	нет	Слишком мало значений: максимум 5 «блоков», популярность жанров неравномерна
(B) readerId	hashed	2500	нет (хеш)	AP-01, AP-05 targeted; запись распределена
(C) loanId	ranged	12000	Да, растёт	Новые записи идут в последний диапазон; AP-01 — scatter-gather
(D) {genreId, readerId} hashed	hashed, составной	4658 (в наборе 80%)	нет	Нужны оба поля в фильтре, иначе scatter-gather
(E) {readerId, day} hashed	hashed, составной	7500	нет	Читатель размазан по дням: AP-01 без day — scatter-gather

Примечание. В базовом наборе genreId = i mod 5, readerId = i mod 2500, а 2500 делится на 5, поэтому жанр однозначно определяется читателем и кардинальность пары {genreId, readerId} равна 2500. В наборе с перекосом 80% жанр назначается случайно, и кардинальность пары вырастает до 4658.

4. Исходный код и запуск

Файл `simulation.py` приложен к отчёту. Запуск: `python simulation.py` (Python 3.10+, только стандартная библиотека; matplotlib нужен лишь для графика). Зафиксирован `random.seed(42)`. Скрипт запускался дважды, вывод побайтно совпал. SHA-256 используется только для учебной симуляции и не воспроизводит внутренний алгоритм MongoDB; распределение по модулю — упрощение, а не настоящий mongos.

```
rows = [dict(reader_id=i % 2500, genre_id=i % 5,
             loan_id=i, day=(i*7) % 30) for i in range(12000)]

def low_card(row):      return row["genre_id"] % 3
def hashed_reader(row): return stable_hash(row["reader_id"]) % 3
def range_loan(row):    return min(row["loan_id"] // 4000, 2)

# hot: 60% запросов к жанру 0, остальные 40% равномерно
hot = [random.choice(hot_rows) if random.random() < 0.6
       else random.choice(rows) for _ in range(30000)]
```

5. Результаты базового эксперимента

Ключ / нагрузка	Документы A/B/C	Запросы A/B/C	Max/mean (запросы)	Max/min (запросы)
genre_id / uniform	4800 / 4800 / 2400	11947 / 12018 / 6035	1.20	1.99
genre_id / hot	4800 / 4800 / 2400	22828 / 4822 / 2350	2.28	9.71
hashed_reader / uniform	3866 / 3805 / 4329	9608 / 9435 / 10957	1.10	1.16
hashed_reader / hot	3866 / 3805 / 4329	9261 / 9774 / 10965	1.10	1.18
range_loan / uniform	4000 / 4000 / 4000	10046 / 9958 / 9996	1.00	1.01
range_loan / hot	4000 / 4000 / 4000	9967 / 9985 / 10048	1.00	1.01

Доли запросов в режиме hot (шарды A/B/C): genre_id — 76.1 / 16.1 / 7.8 %; hashed_reader — 30.9 / 32.6 / 36.5 %; range_loan — 33.2 / 33.3 / 33.5 %.

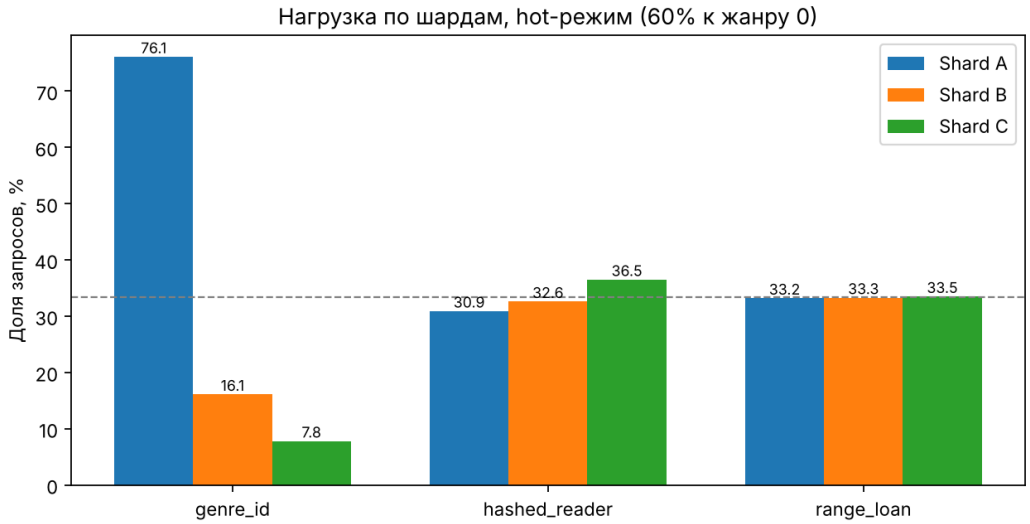


Рисунок 1. Доля запросов по шардам в режиме hot.

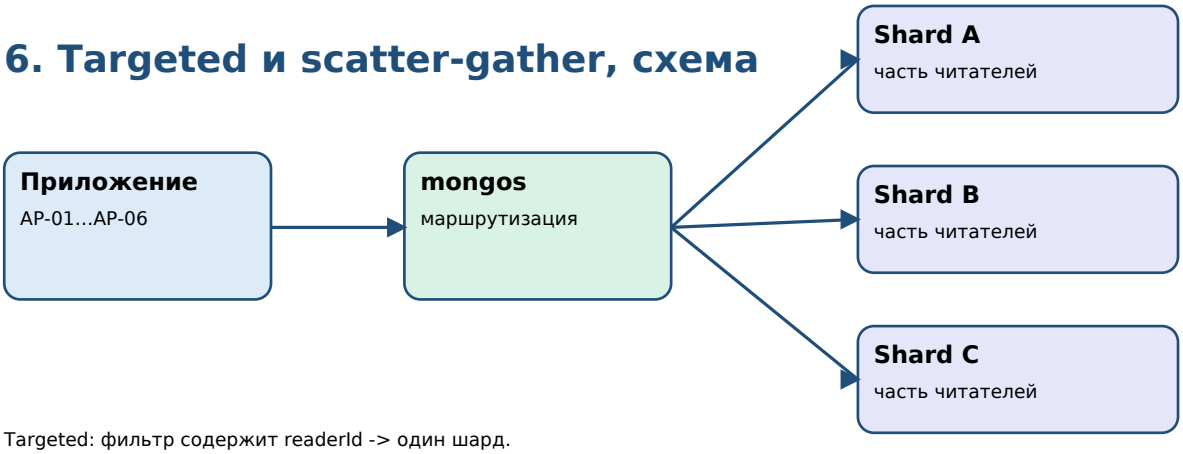
Интерпретация

genre_id. Документы распределены 4800/4800/2400 (max/mean 1,2), но при 60% запросов к жанру 0 шард A получает 76.1% нагрузки (max/mean 2.28, max/min 9.71). Равномерность по объёму не означает равномерность по нагрузке: нагрузку определяет популярность значений ключа.

hashed_reader. Хеширование даёт умеренный разброс (max/mean около 1,1 из-за малого числа значений: 2500 читателей на 3 шарда) и почти не реагирует на перекос по жанру, так как жанр не входит в ключ.

range_loan. Документы и запросы почти идеально равномерны, но это артефакт теста: запросы выбирают случайные существующие документы, а жанр не коррелирует с loanId. Слабость ranged-ключа проявляется при записи (раздел 6, AP-04).

6. Targeted и scatter-gather, схема



Targeted: фильтр содержит readerId -> один шард.
Scatter-gather: фильтр без readerId -> все шарды.

AP	(A) genreId	(B) readerId hashed	(C) loanId ranged
AP-01 выдачи читателя	scatter-gather	targeted	scatter-gather
AP-02 интервал времени	scatter-gather	scatter-gather	scatter-gather (loanId не равен дате; при корреляции — targeted по диапазону)
AP-03 агрегат по жанру	targeted (один жанр) / scatter для всех	scatter-gather	scatter-gather
AP-04 новый поток (запись)	по жанрам, возможен перекос	равномерно	всё в один (последний) шард
AP-05 просрочки читателя	scatter-gather	targeted	scatter-gather
AP-06 жанр за неделю	targeted	scatter-gather	scatter-gather

Эксперимент AP-04: новый поток из 1000 выдач (loanId = 12000…12999) распределился по шардам A/B/C так: genre_id — 400 / 400 / 200; hashed_reader — 304 / 340 / 356; range_loan — 0 / 0 / 1000. Монотонный ключ направляет всю запись на один шард (горячий шард по записи).

7. Улучшение модели: 80% записей в одном жанре

Доля жанра 0 в наборе — 80.1%; в запросах 80% обращений идут к этому жанру.

Ключ	Документы A/B/C	Запросы A/B/C	Max/mean (док.)	Max/mean (запр.)
genre_id	10213 / 1200 / 587	29070 / 610 / 320	2.55	2.91
hashed_reader	3866 / 3805 / 4329	9763 / 9512 / 10725	1.08	1.07
hashed{genre, reader}	3945 / 4117 / 3938	9927 / 10270 / 9803	1.03	1.03
hashed{reader, day}	4072 / 3969 / 3959	10122 / 9778 / 10100	1.02	1.01

Сравнение стратегий и аргументы выбора

Сравниваются текущий кандидат {readerId hashed} и альтернатива {readerId, day_bucket} hashed (или {genreId, readerId} hashed).

1. Нагрузка. При 80% перекосе по жанру ключ genre_id даёт max/mean 2.91 по запросам (shard A принимает 96.9% запросов), а ключи на основе readerId держат max/mean в пределах 1,01–1,07.

2. Кардинальность. readerId даёт 2500 значений, {readerId, day} — 7500, {genreId, readerId} — 4658: чем больше значений, тем мельче чанки и тем легче балансировщику.

3. Targeted-запросы. readerId обслуживает AP-01 и AP-05 одним шардом. Составные ключи требуют второго поля в фильтре (genreId или day), иначе запрос превращается в scatter-gather.

4. Риск «горячего» читателя. Один очень активный читатель всегда попадёт на один шард при ключе readerId. Добавление day_bucket дробит такого читателя по дням, но ценой scatter-gather для его полной истории.

5. Запись. Hashed-ключ равномерно принимает новый поток (AP-04), ranged по loanId — нет.

Вывод по сравнению: для основного сценария readerId (hashed) лучше; составной {readerId, day_bucket} стоит вводить только при подтверждённых активных читателях-выбросах.

8. Дополнительное задание (кластер MongoDB)

Реальный шардированный стенд не использовался: он недоступен, а обычный одиночный mongod не является кластером. Для стенда с правами администратора предназначены команды (подключение через mongos):

```
use university_lab
db.loans.createIndex({readerId: "hashed"})
sh.shardCollection("university_lab.loans", {readerId: "hashed"})
db.loans.getShardDistribution()
db.loans.find({readerId: 1001}).explain("executionStats")
```

Ожидание: в плане запроса по readerId — один шард (targeted). Числа реального кластера не совпадут с моделью из-за чанков и балансировщика MongoDB.

9. Заключение

Рекомендуемый shard key: `{readerId: "hashed"}`.

Компромиссы. Hashed-ключ даёт ровную запись и targeted-доступ для AP-01 и AP-05, но делает диапазонные запросы AP-02 и агрегаты AP-03/AP-06 scatter-gather. Если отчёты тяжёлые, их стоит выносить в отдельную аналитическую коллекцию или реплику.

Риски. (1) Горячий читатель или горячая книга; (2) рост числа запросов без readerId ухудшит масштабирование; (3) shard key нельзя менять без учёта ограничений версии MongoDB — стоит проверять по документации.

Ограничения модели. Распределение по модулю и SHA-256 не воспроизводят чанки, зоны и балансировщик MongoDB; данные синтетические; запросы считаются равными по стоимости; сетевые задержки и размер документов не моделируются; в базовом наборе жанр функционально зависит от читателя.

10. Ответы на контрольные вопросы

21. Чем шардирование отличается от репликации? Шардирование делит данные между шардами (масштаб по объёму и нагрузке), репликация копирует одни и те же данные на несколько узлов (доступность).

22. Что такое shard key? Поле (или поля), по которому СУБД размещает документы и маршрутизирует запросы. Его выбор критичен: он определяет баланс, targeted-запросы и трудоёмкость смены.

23. Кардинальность и риск низкой? Число различных значений ключа. При малом числе значений нельзя разбить данные на достаточно чанков; здесь genreId (5 значений) дал max/mean по запросам 2.28.

24. Когда ranged удобнее hashed? Когда преобладают диапазонные запросы по ключу и ключ не монотонен.

25. Почему монотонный ключ создаёт горячий шард? Новые значения всегда в конце диапазона, поэтому все вставки идут в один шард (в AP-04: 0/0/1000).

26. Targeted и scatter-gather? Targeted — запрос направлен на один шард по ключу в фильтре; scatter-gather — запрос рассылается на все шарды и результаты объединяются mongos.

27. Почему равномерные документы не дают равномерной нагрузки? Нагрузка зависит от популярности значений: при 60% запросов к одному жанру шард A получил 76,1% запросов при 40% документов.

28. Как читать max/mean? Отношение нагрузки самого загруженного шарда к средней: 1,0 — идеал, большие значения — перекос и риск горячего шарда.

29. Ограничения Python-симуляции? Нет чанков, балансировки, зон, сети, индексов и реального mongos; хеш и модуль — упрощения.

30. Что измерить перед внедрением? Профиль запросов (доли и фильтры), рост данных, распределение значений кандидатов, пиковую нагрузку, долю запросов без ключа, размер документов и индексов.